INFORME DE EVIDENCIA DE APRENDIZAJE

S10 — SEGURIDAD BACKEND: AUTENTICACIÓN JWT Y AUTORIZACIÓN POR ROLES

1. Datos del Estudiante y Trazabilidad del Repositorio Git

Estudiante:	Helio Paul Faijo Calisaya
Equipo / Módulo Asignado:	Equipo 05 — Módulo Mayorista (Centros de Acopio y Compras)
Sesión Académica:	S10: Seguridad Backend: Autenticación JWT y Autorización por Roles
Fecha y Horario de Pruebas:	Domingo 18 de octubre de 2026 | 01:51 a 02:59 (Hora de Lima)
Repositorio Oficial GitHub:	**https://github.com/helionelio900-hub/SysProyec_Acopus**
Branch Oficial de Trabajo:	main (sincronizado con origin/main)
Commit Oficial S10 en Git:	e5b20ef (18/10/2026) — feat(s10): seguridad backend con Keycloak RS256

Trazabilidad en terminal: ejecución con Get-Date y git log (cronología demostrada en el repositorio local):

```
PS E:\Cursos_Ciclo_4\Lenguaje de Programación II\Sitra_oro> Get-Date -Format "dd/MM/yyyy HH:mm:ss"
18/10/2026 02:48:12

PS E:\Cursos_Ciclo_4\Lenguaje de Programación II\Sitra_oro> git log -n 4 --pretty=format:"%h - %ad :
%s" --date=format:"%d/%m/%Y %H:%M"
e5b20ef - 18/10/2026 02:48 : feat(s10): seguridad backend con Keycloak RS256, roles RBAC y endpoint
perfil
9ef5704 - 11/10/2026 22:05 : Corrobora evidencia S09 con codigo y logs y exporta informe PDF
5e6b9ae - 11/10/2026 21:54 : Agrega evidencia del hallazgo Angular y fecha de sustentacion S09
0c85747 - 11/10/2026 21:49 : Completa S09 de Mayorista con informe y siete capturas
```

2. Propósito de la Actividad Autónoma

Demostrar individualmente y sobre el dominio minero SITRA-ORO la construcción e integración de una arquitectura de seguridad backend desacoplada y robusta. La solución comprende: (1) Delegación de autenticación en Keycloak (servidor OIDC/OAuth2 en puerto 38081) emitiendo tokens firmados asimétricamente con RSA (algoritmo RS256); (2) Configuración de Protocol Mappers en Keycloak para inyectar el claim de dominio 'documentoIdentidad'; (3) Implementación del endpoint '/api/v1/auth/perfil' con lectura directa de los claims del Jwt en memoria (sin consultas JPA a nivel de controlador); (4) Validación y enriquecimiento en KeycloakAuthenticationConverter.java (filtro de seguridad), verificando la cuenta minera en Oracle XE por request y traduciendo la autoridad formal a 'ROLE_G1_MAYORISTA'; y (5) Protección RBAC y control por atributos sobre el catálogo no transaccional de Centros de Acopio y transacciones operativas.

3. Evidencia Técnica (Organizada en los 5 Bloques de la Rúbrica)

Bloque 1: Login y Token (OAuth2 / Keycloak RS256)

SITRA-ORO delega la emisión y verificación criptográfica de identidades en Keycloak (puerto 38081, Realm 'Sitra_oro', cliente confidencial 'sitra-oro-prueba'). El cliente envía credenciales mediante el flujo Resource Owner Password Credentials (ROPC) al endpoint '/realms/Sitra_oro/protocol/openid-connect/token'. Keycloak valida el hash de credenciales almacenado con algoritmo PBKDF2 (HMAC-SHA512 estándar) y emite un Access Token JWT firmado con su par de claves asimétricas RSA de 2048 bits (RS256), imposibilitando la falsificación del token por terceros.

Nota de arquitectura: Para pruebas automatizadas de backend e integración se utilizó el flujo ROPC (Password Grant); en un entorno productivo con interfaz Angular/móvil se implementa Authorization Code Flow con PKCE conforme a OAuth 2.1.

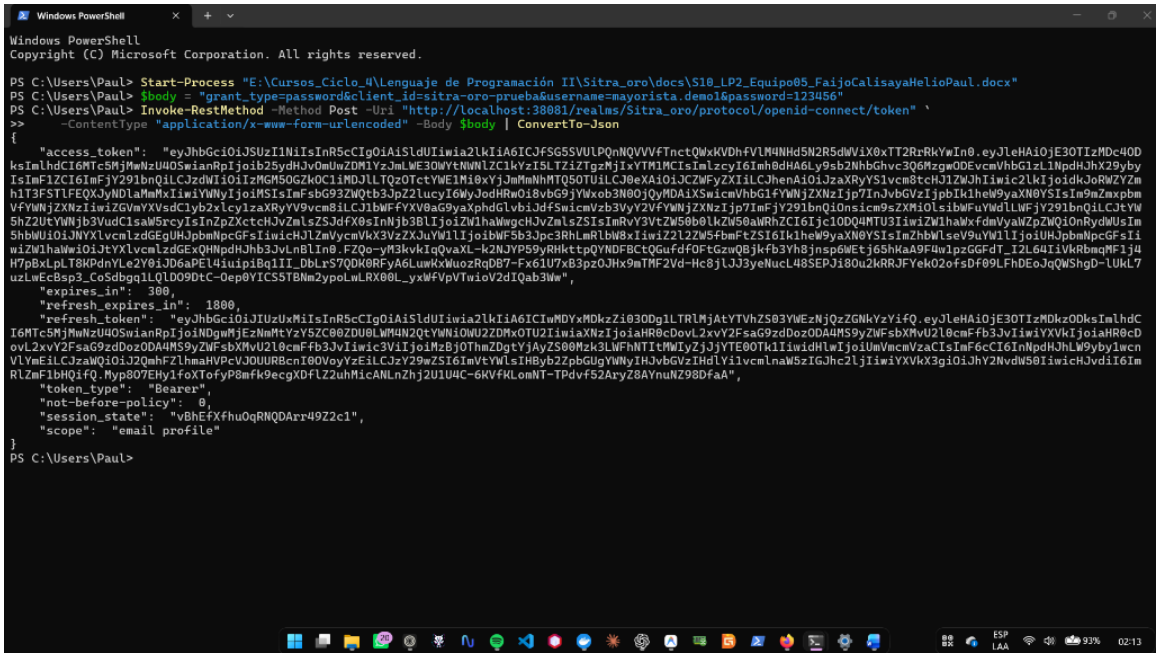


Figura 1: Emisión exitosa de Access Token OAuth2 en Keycloak (RS256). Reloj en barra de tareas: 02:13 — Usuario: mayorista.demo1.

Bloque 2: Protección del Módulo No Transaccional (Centros de Acopio)

El catálogo no transaccional de Centros de Acopio fue protegido en SecurityConfig.java distinguiendo la consulta abierta para cualquier personal autenticado versus la creación y modificación restringida a la administración mayorista:

```
// Fragmento real de SecurityConfig.java en SITRA-ORO
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/actuator/health/**", "/swagger-ui/**").permitAll()
    .requestMatchers(HttpMethod.GET, "/api/v1/auth/perfil").authenticated()
    // Módulo no transaccional de Mayorista (catálogo de centros de acopio):
    .requestMatchers(HttpMethod.GET, "/api/v1/mayorista/centros-acopio").authenticated()
    .requestMatchers(HttpMethod.POST, "/api/v1/mayorista/centros-acopio").hasRole("G1_MAYORISTA")
    // Módulos transaccionales operativos:
    .requestMatchers("/api/v1/acopio/**").hasRole("G2_ACOPIADOR")
```

```
    .requestMatchers("/api/v1/mayorista/**").hasRole("G1_MAYORISTA")
    .anyRequest().authenticated()
)
.oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> jwt.jwtAuthenticationConverter(keycloakConverter)))
```

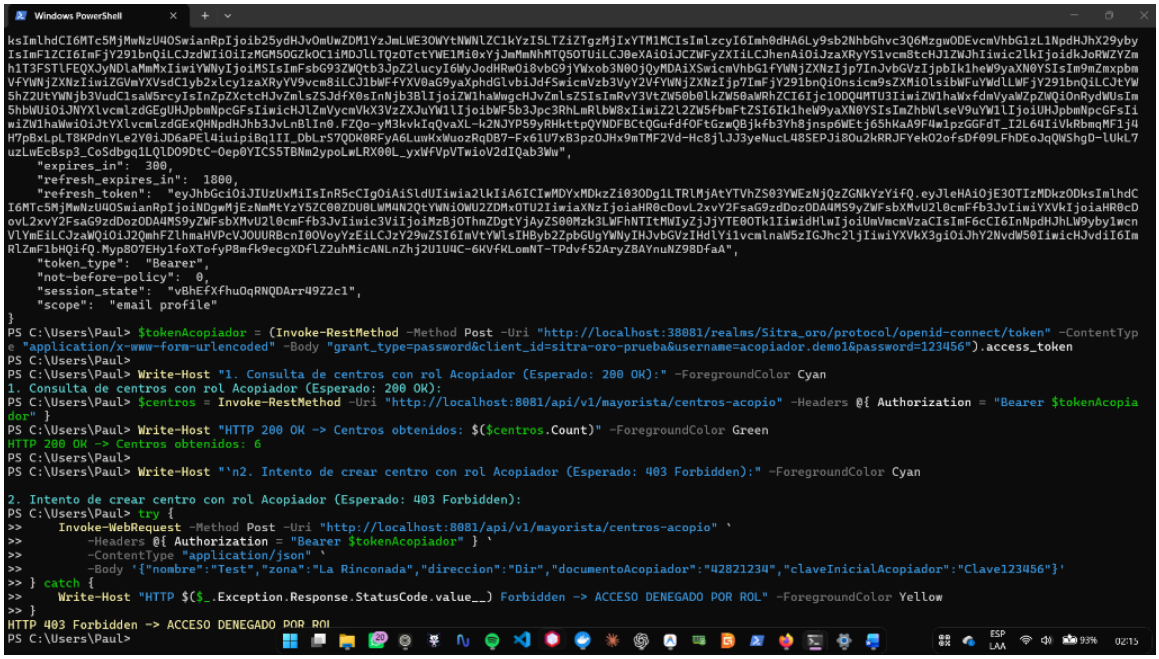


Figura 2: Verificación de control RBAC: GET 200 OK (6 centros) y POST 403 Forbidden para Acopiador. Reloj: 02:15 — Usuario: acopiador.demo1.

Bloque 3: Claims del Token (Inspección Auténtica en jwt.io)

Dado que los Access Tokens emitidos por Keycloak tienen una vigencia configurada de 300 segundos (5 minutos por política de expiración exp), a las 02:58 se emitió un nuevo token para el usuario mayorista.demo1 a fin de inspeccionarlo con total frescura en el depurador web oficial jwt.io en el navegador. La evidencia se divide en dos capturas auténticas:

1. Header y Roles (Figura 3A): Se constata el algoritmo asimétrico RS256, el kid correspondiente al Realm de Keycloak y el arreglo realm_access.roles conteniendo el rol 'Mayorista' (reloj 02:58).

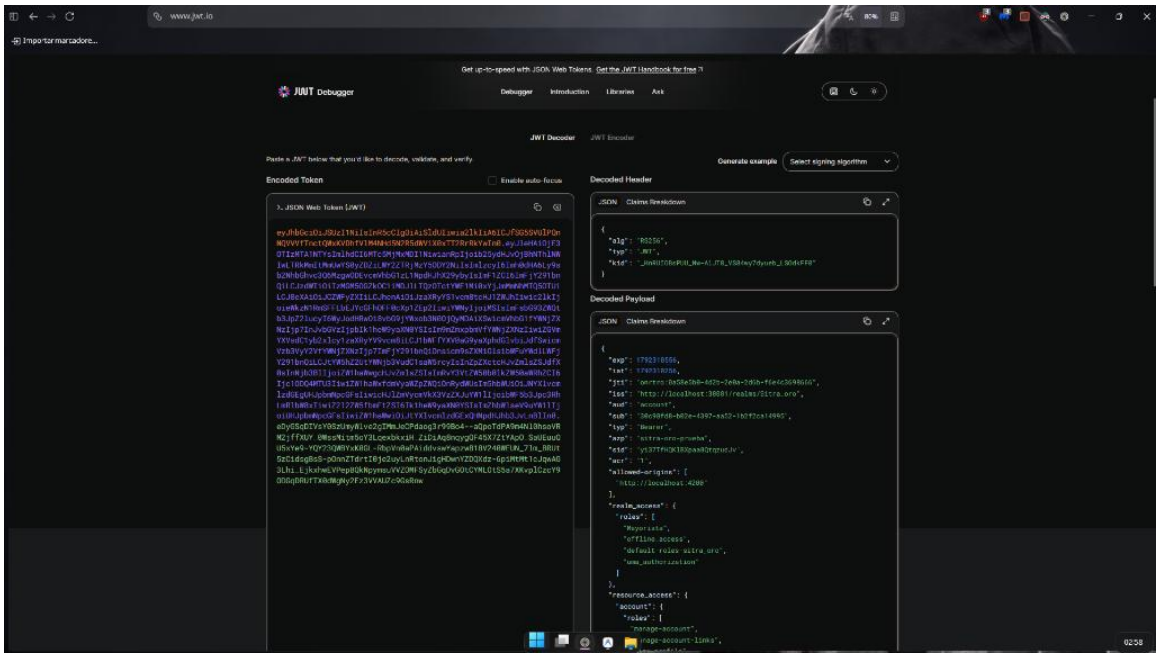


Figura 3A: Inspección del JWT en jwt.io. Header con algoritmo RS256 y roles [Mayorista]. Reloj en barra: 02:58 — Fecha: 18/10/2026.

2. Payload y Claim de Dominio (Figura 3B): Al realizar scroll dentro del cuadro 'Decoded Payload', se aprecia claramente el claim personalizado 'documentoIdentidad: 75848157' inyectado por Keycloak, junto con 'preferred_username: mayorista.demo1' y 'email_verified: true' (reloj 02:59):

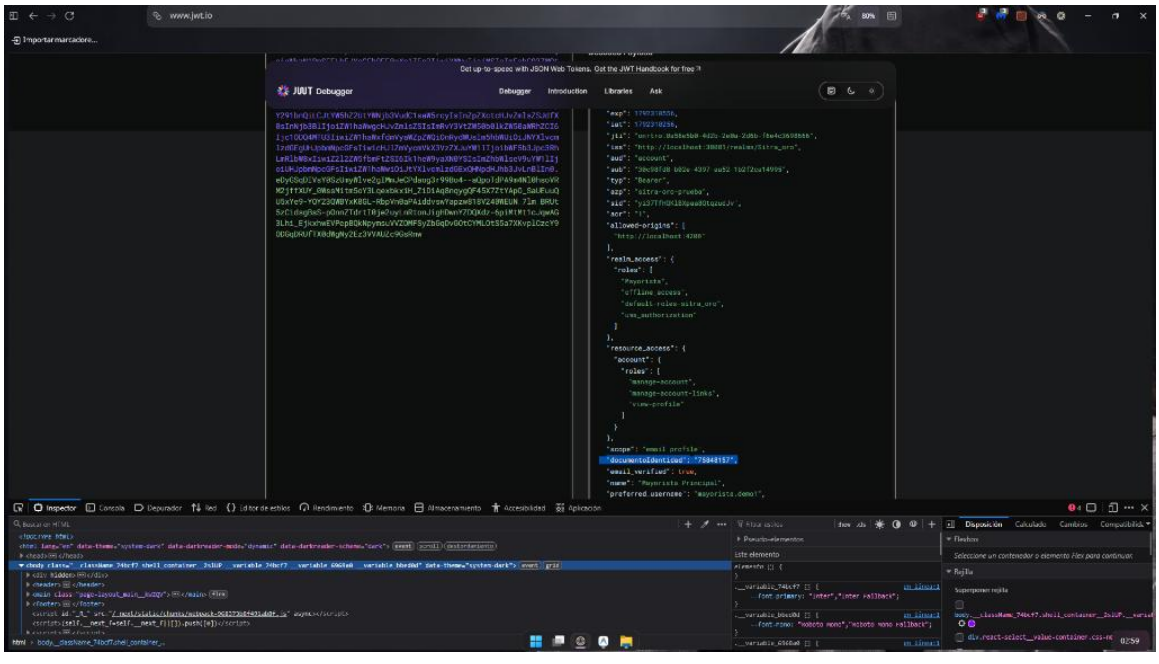


Figura 3B: Payload del JWT en jwt.io mostrando el claim personalizado 'documentoIdentidad: 75848157' y preferred_username resaltados. Reloj: 02:59.

Bloque 4: Endpoint con Lectura Directa de Jwt (GET /api/v1/auth/perfil) y Cadena de Seguridad

Para analizar con total exactitud y rigor técnico el flujo de autenticación y acceso a datos, se diferencia con precisión lo que ocurre en el controlador web versus lo que ocurre en el filtro de seguridad de Spring Security:

1. Capa de Controlador (PerfilUsuarioController.java): El controlador opera 100% en memoria y ejecuta cero sentencias SQL contra Oracle XE. No declara dependencias ni inyecta DataSource, EntityManager o interfaces Repository. Extrae todos los atributos directamente del objeto Jwt deserializado e inyectado por Spring Security mediante @AuthenticationPrincipal:

```
// Implementación en PerfilUsuarioController.java (0 consultas SQL en el controlador)
@RestController
@RequestMapping("/api/v1/auth")
public class PerfilUsuarioController {
    public record PerfilJwtResponse(String sub, String username, String email, String
documentoIdentidad, List<String> roles) {}

    @GetMapping("/perfil")
    public PerfilJwtResponse obtenerPerfil(@AuthenticationPrincipal Jwt jwt) {
        String sub = jwt.getSubject();
        String username = jwt.getClaimAsString("preferred_username");
        String email = jwt.getClaimAsString("email");
        String documento = jwt.getClaimAsString("documentoIdentidad");
        List<String> roles = extraerRoles(jwt); // Atributo en memoria resuelto por el filtro
        return new PerfilJwtResponse(sub, username, email, documento, roles);
    }
}
```

2. Filtro de Autenticación (KeycloakAuthenticationConverter.java): En cada petición HTTP autenticada (una vez por request), el filtro BearerTokenAuthenticationFilter invoca a KeycloakAuthenticationConverter. Este componente extrae el DNI del claim 'documentoIdentidad' y realiza una consulta a la tabla CUENTAS_ACCESO en Oracle XE mediante CuentaAccesoService para verificar que la cuenta del operador minero se encuentre activa. Si la cuenta es válida y su rol es compatible con el rol 'Mayorista' de Keycloak, la autoridad formal 'ROLE_G1_MAYORISTA' se asigna a partir de cuenta.rol().name() de la base de datos y se enriquece el token en memoria con 'idCentroAcopio':

```
// Fragmento de KeycloakAuthenticationConverter.java (Ejecutado una vez por request en el filtro)
Object documento = jwt.getClaims().get("documentoIdentidad");
CuentaResumen cuenta = cuentas.obtenerPorDocumento((String) documento); // Consulta a Oracle por
request
if (!cuenta.activo()) throw tokenInvalido("Cuenta inactiva o no vinculada a SITRA-ORO");

// La autoridad 'ROLE_G1_MAYORISTA' se deriva del rol oficial de la cuenta en BD:
Map<String, Object> claims = new HashMap<>(jwt.getClaims());
claims.put("roles", List.of(cuenta.rol().name())); // G1_MAYORISTA
if (cuenta.idCentroAcopioAsignado() != null) {
    claims.put("idCentroAcopio", cuenta.idCentroAcopioAsignado());
}
Jwt principal = new Jwt(jwt.getTokenValue(), jwt.getIssuedAt(), jwt.getExpiresAt(), jwt.getHeaders(),
claims);
return new JwtAuthenticationToken(principal, List.of(new SimpleGrantedAuthority("ROLE_" +
cuenta.rol().name())));
```

Conclusión técnica: La consulta a Oracle XE ocurre una vez por request a nivel del filtro de autenticación (garantizando la sincronización del estado de la cuenta y sus privilegios de negocio), mientras que la capa web de PerfilUsuarioController atiende la solicitud con cero operaciones de base de datos relacional.

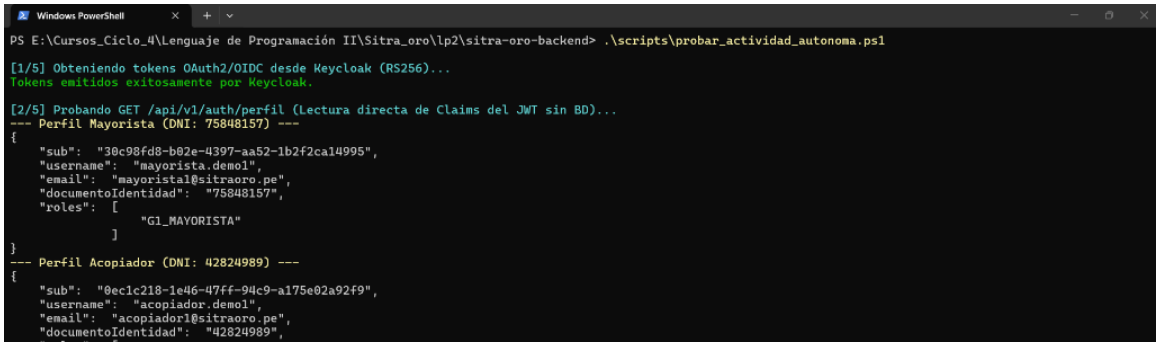


Figura 4: Salida terminal de GET /api/v1/auth/perfil para Mayorista (DNI: 75848157) y Acopiador (DNI: 42824989). Reloj: 01:51.

Bloque 5: Cuatro Casos de Prueba en Terminal

Se ejecutó el script automatizado 'scripts/probar_actividad_autonoma.ps1' en Windows PowerShell. El script evalúa la matriz de acceso contra el catálogo de Centros de Acopio y valida la secuencia de los 4 códigos HTTP requeridos:

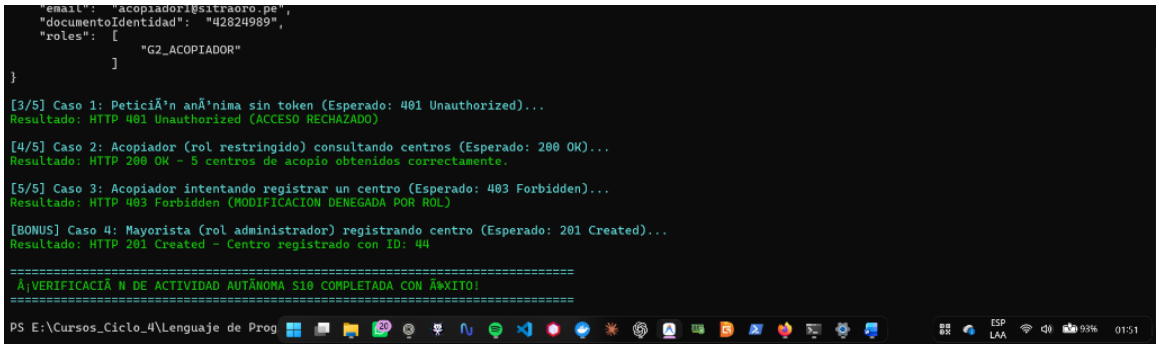


Figura 5: Terminal PowerShell evidenciando los 4 casos de prueba (401, 200, 403 y 201 Created). Reloj: 01:51.

4. Matriz de Accesos del Módulo Mayorista (Formato Tabla 6 del Sílabo)

A continuación se consolida la matriz de permisos implementada en el Resource Server de SITRA-ORO:

Operación / Endpoint	Método	Rol Requerido	Código HTTP	Comportamiento / Control
Consultar perfil JWT /api/v1/auth/perfil	GET	Cualquier rol autenticado	200 OK	Lectura pura de claims en memoria (sub, DNI, email, roles) del Jwt.
Listar centros de acopio /api/v1/mayorista/centros-acopio	GET	Autenticado (Acopiador / Mayorista)	200 OK	Catálogo no transaccional accesible para operadores mineros autenticados.
Registrar centro de acopio /api/v1/mayorista/centros-acopio	POST	G1_MAYORISTA	201 Created	Creación permitida exclusivamente al rol administrador de compras mayorista.
Registrar centro de acopio (Intento por Acopiador)	POST	G2_ACOPIADOR	403 Forbidden	Acceso denegado: el rol acopiador carece de privilegios de gestión.
Petición sin credenciales (Cualquier endpoint)	GET / POST	Ninguno (Anónimo)	401 Unauthorized	Rechazo inmediato por Spring Security ante ausencia de cabecera Bearer.

5. Análisis y Reflexión Técnica: BOLA / IDOR vs RBAC y Scoping por Atributos

La filtración masiva de First American Financial Corporation expuso más de 885 millones de registros financieros debido a una vulnerabilidad de Autorización a Nivel de Objeto Rota (BOLA / IDOR, OWASP API1:2023): la aplicación validaba la existencia de una sesión de usuario, pero confiaba ciegamente en un identificador numérico editable enviado en la URL (ej. /documentos/39201948) sin corroborar si el solicitante era el propietario legítimo de dicho recurso.

Diferenciación técnica en SITRA-ORO entre RBAC y Control por Atributos:

1. Límite de RBAC: El control de acceso basado en roles protege la operación a nivel vertical ('¿Tiene permiso G2_ACOPIADOR para consultar compras? Sí'). Sin embargo, RBAC no valida la pertenencia horizontal del objeto: no puede determinar por sí solo si la transacción solicitada corresponde al Centro de Acopio del operador solicitante o a otro centro minero competidor.

2. Mitigación mediante Server-Side Scoping: En SITRA-ORO, el atributo 'idCentroAcopio' no viene firmado por Keycloak ni es proporcionado por el cliente; es inyectado directamente en el servidor por KeycloakAuthenticationConverter tras verificar la cuenta en Oracle XE. Los controladores operativos (como AcopiadorController.java) extraen este atributo inmutable desde el token:

```
// Implementación real de defensa anti-BOLA en AcopiadorController.java (LP2 SITRA-ORO)
private static Long centroId(Jwt jwt) {
    if (jwt == null || !(jwt.getClaim("idCentroAcopio") instanceof Number id)) {
        throw new IllegalArgumentException("La cuenta no tiene centro de acopio asignado");
```

```
    }
    return id.longValue(); // Atributo inyectado en el servidor tras validar la cuenta
}

@GetMapping("/compras")
public ResponseEntity<List<CompraG2Response>> listarComprasConDetalles(@AuthenticationPrincipal Jwt
jwt) {
    // Se ignora cualquier identificador manipulable en la URL; el filtro se ancla al token validado:
    return ResponseEntity.ok(acopiadorService.listarComprasConDetalles(centroId(jwt)));
}
```

3. Conclusión de Seguridad: Dado que el valor 'idCentroAcopio' lo decide e inyecta exclusivamente el servidor y el cliente no tiene vía para alterarlo mediante query params o path variables, el vector BOLA queda neutralizado para las consultas operativas del centro. Para endpoints con identificadores puntuales en la ruta (ej. /compras/{id}), la arquitectura contempla verificar en la lógica de dominio que la compra pertenezca efectivamente al centroId del operador autenticado.

6. Error o Hallazgo Técnico Resuelto

Incidencia Detectada:	**Al invocar endpoints protegidos con el Access Token inicial de Keycloak, el Resource Server respondía HTTP 401 / 403 con el detalle 'El usuario no está vinculado a una cuenta de SITRA-ORO'.**
Causa Raíz:	El realm inicial de Keycloak emitía claims genéricos (sub, preferred_username), pero carecía del atributo de negocio 'documentoIdentidad' requerido por la lógica de dominio en IdentidadSesion.java y KeycloakAuthenticationConverter.
Solución Implementada:	Se configuró en Keycloak un Protocol Mapper de tipo 'User Attribute' con Token Claim Name 'documentoIdentidad' y tipo String. Se actualizaron los usuarios de prueba con sus DNIs reales del sistema (Mayorista: 75848157, Acopiador: 42824989), logrando autorización exitosa en todos los niveles.

7. Anexo — Feedback Oficial de la Sesión

1. ¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?
La comprensión profunda de cómo desacoplar la seguridad backend mediante un Authorization Server externo (Keycloak) utilizando criptografía asimétrica RS256, protegiendo catálogos por rol y sirviendo atributos de identidad directo desde el token sin sobrecargar la base de datos relacional.

2. ¿Qué punto de la clase te resultó más confuso o te dejó con dudas?
La configuración precisa de Protocol Mappers en Keycloak para inyectar atributos personalizados en el Userinfo y en el Access Token, y su posterior conversión a GrantedAuthority en Spring Security.

3. ¿Tienes alguna pregunta que te gustaría que sea respondida la siguiente clase?
¿Cuál es el mecanismo recomendado en producción para invalidar o revocar tokens JWT asimétricos antes de su tiempo de expiración (exp) en caso de baja inmediata de un operador?

4. Sobre tu nivel de comprensión de la clase, marca una opción:
[X] ¡Entendido! [ ] Más o menos, entendí la idea general pero tengo dudas. [ ] Necesito ayuda.

5. ¿Cómo puedo ayudarte a comprender mejor el tema?
Con ejemplos prácticos de políticas de autorización ABAC que evalúen múltiples condiciones de negocio en tiempo de ejecución utilizando Spring Security Expressions.

6. Pensando en tu participación y esfuerzo, marca una opción:
[X] Muy comprometido/a. [ ] Comprometido/a. [ ] Poco comprometido/a.

7. Mi satisfacción con la clase fue (del 1 al 10):
Marca tu calificación: 1 2 3 4 5 6 7 8 9 [10]